Input validation with Hibernate and regular expressions

Notes

Goal of the chapter

This chapter discusses one of the most fundamental secure coding techniques: *input validation*.

Acknowledgement and further reading

The content of this section has been extracted from

- `Iron-CladJava:BuildingSecureWebApplications(OraclePress) 1st~Edition~~JimManico,~August~Detlefsen.`
- `https://hibernate.org/validator/`
- `https://owasp-top-10-proactive-controls-2018.readthedocs.io/en/latest/c5-validate-all-inputs.html`

1 Data Validation and encoding

Adversaries often gain a foothold into software application by sending cleverly constructed inputs to negatively influence a program's behaviours. Input that is inconsistent with expected functionality and design will often cause the application to act in unexpected ways. Garbage in, pure exploitation gold out.

Adversaries hack your web application by adding attack strings to legal request: The HTTP protocol itself is not made invalid by these attacks. Input validation is a programming technique that limits what input an application will legally accept. Proper input validation defines what data is allowed and rejects the rest. After passing validation, data can then be processed by your application's business logic layer. This is one of several important layers of defence needed in a secure web application.

1.1 Blacklisting (vulnerable)

Blacklist or negative validation defines what an attack looks like and blocks it. This is an interesting and sometimes effective form of intrusion detection, but it is not a strong defence. Blacklist validation may stop some very dangerous well-known attaccks to your application, but it is not a control that will stop all attacks. There is a huge incentive and relatively little risk for hackers to try new things. This means that the threat landscape will be constantly evolving and there are essentially an infinite number of possible attack patterns to block. There are many types of *filter evasion techniques* such as encoding your attacks or submitting legal input that might contain an attack (like the SQL injection email address `' --@manico.net`. Commercial fuzzers exist that can constantly bombard your applications with tiny variations of inputs until a weakness is found. These alone can make maintaining a blacklist a daunting chore, at best.

1.2 Whitelisting (secure)

Rather than starting by attempting to block characters that are known to be bad (blacklisting), we should instead begin by defining only those characters and patterns that are known to be good (whitelisting). For example, a phone number input field should only need to accept digits. Even if we allow users to submit formatted phone numbers, a very limited range of characters are acceptable.

Hint

Deny by default. Reject all characters except for specific characters of interest.

2 Input validation with Hibernate

Hibernate Validator allows to express and validate application constraints. The default metadata source are annotations, with the ability to override and extend through the use of XML. It is not tied to a specific application tier or programming model and is available for both server and client application programming. Consider the following example code from this 's web app:

```
public class Client {

 @NotNull(message = "is required")
 @Size(min = 3, message="is required")
 //@Pattern(regexp =".*[^<>$]", message="incorrect format")
 @Pattern(regexp ="[a-zA-Z]+", message="incorrect format")
 protected String name;

 //@NotNull(message = "is required")
 //@Min(value=18, message = "You must be 18 years old or older")
 //@Max(value=120, message = "Vampires are not allowed")
 protected int age;



 public String getName() {
  return name;
 }
 public void setName(String name) {
  this.name = name;
 }
 public int getAge() {
  return age;
 }

   // ...
}
```

2.1 Force required fields and validate number ranges

Let us assume we would like to enforce the following validation rules.

- The client's name should not be empty
- Their age should be higher than 17 and lower than 120.

Hibernate and Spring allow us to this in a fairly simple way. We need to execute the following steps:

1. **Use annotations to establish validation rules.**
    - Go to our Client class. This is our model, and it contains the attributes we would like to validate.
    - Add the following annotations:

```
1
2    //import this library
3    import javax.validation.constraints.*;
4
5  @NotNull(message = "is required")
6  @Size(min = 1, message="is required")
7  protected String name;
8
9  @Min(value=18, message = "You must be 18 years old or older
       ")
10 @Max(value=120, message = "Vampires are not allowed")
11 protected int age;
12
```

The annotations are self-explanatory. The message attribute within the annotation will be used by the view layer to display the message. How? We will see that later. Prior to that let's tell our controller to do input validation.

- **Tell our controller to perform validation.**
  - Go to the class CoachController.
  - Modify the workout method as follows:

```
1
2
3     //impor these libraries
4     import javax.validation.Valid;
5
6     import org.springframework.validation.BindingResult;
7
8
9  public String workout(
10   @Valid @ModelAttribute("client") Client client,
11   BindingResult validationErrors, Model model) {
12  //Input validation
13  if (validationErrors.hasErrors())
14   return "client-form";
15  //Logic when there is no error
16  .
17  .
18  .
19 }
20
```

  - Notice that we have used the annotation `@Valid` before the annotation `@ModelAttribute`. This is to tell spring to perform input validation on the object client.
  - Also notice that we adding a new argument to the method with the object `BindingResult validationErrors`. Spring will create this object for us. It contains information about the validation process, as we will see next.
  - The conditional `validationErrors.hasErrors()` is used in our controller class to load again the client-form.jsp page if something

went wrong.

Important information

Spring requires the argument `BindingResult validationErrors` to be place right after the `@ModelAttribute` argument.

2. **Add an error message to our form JSP page.**

   - Go to client-form.jsp
   - Add the Spring tag `<form:errors ...` to the form as follows.

```
<form:form action="processForm" modelAttribute="client" >
 Name (*): <form:input path="name" />
 <form:errors path="name" cssClass="error" />

 <br><br>

 Age (*): <form:input path="age" />
 <form:errors path="age" cssClass="error" />

 <br><br>

 <input type="submit" value="Submit" />
</form:form>

```

   The Spring tag form `<form:errors ...` renders field errors in an HTML 'span' tag. It provides access to the errors created in your controller or those that were created by any validators associated with your controller. In our case, the attribute `message` that we are using in our annotations in the Client class will be displayed by this form. Also notice the attribute `path="name"` in the form tag. This binds the form to the variable name within the object client.

   - Because we are using a cssClass, let's add it to the .jsp page.

```
<head>
<title>Client Registration Form</title>

 <!--  Inline CCS -->
 <style>
  .error {color:red}
 </style>

</head>

```

Some of the annotations that are defined in Java Bean Validation API are as follows:

- `@NotBlank`- The annotated element must not be null and must contain at least onenon-whitespace character.
- `@NotEmpty`- The annotated element must not be null nor empty.
- `@NotNull`- The annotated element must not be null.
- `@Size`- The annotated element size must be between the specified boundaries. Boundaries can be specifies using main and max attributes.
- `@Digits`- The annotated element must be a number within accepted range.
- `@Max`- The annotated element must be a number whose value must be lower orequal to the specified maximum.
- `@Min`- The annotated element must be a number whose value must be higher orequal to the specified minimum.
- `@Email`- The string has to be a well-formed email address.

2.2 Validate using regular expressions

A regular expression is a language that defines a text pattern. It is important that you understand them.

We will follow similar steps to those used earlier.

1. **Use annotations to establish validation rules.** In this case we are using regular expressions.
   - Go to our Client class. Rather than using the `@NotNull` and `@Size` annotations, let's use the `@Pattern` annotation. Our code should look as follows.

```
@Pattern(regexp="^[A-Z][a-z]*", message="A name should
    start with a "
   + "capital letter and be followed by lower case letters"
    )
protected String name;

```

2. **Tell our controller to perform validation.** We already have this implemented. No further changes need to be made.
3. **Add an error message to our form JSP page.** Already done.